МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ СИСТЕМ

Лабораторна робота №2

Виконала:
студентка 4-го курсу
денної форми навчання
групи ІПЗ-1
Граб Емілія Володимирівна

Ужгород - 2026

1. Титульний аркуш, мета роботи, номер варіанта

Тема: Модульне тестування бізнес-логіки засобами Vitest. Матчери, параметризовані тести, покриття коду.

Мета: Набути навичок розроблення модульних тестів для бізнес-логіки: добирати матчери відповідно до типу даних, проєктувати вхідні дані за класами еквівалентності та межовими значеннями, застосовувати параметризовані тести, вимірювати й інтерпретувати покриття коду.

Варіант 12

Індивідуальне завдання: calcInstallment(total, months) — місячний платіж для 3, 6 та 12 місяців із надбавкою 2 % за кожні три місяці.

Титульний аркуш і дані студентки збережено. Відсутні чотири функції показового прикладу додано за оригінальною методичкою з форматуванням Prettier. Наявну реалізацію варіанта 12 збережено.

Фінальні перевірки виконано 19.10.2026 (Europe/Uzhgorod): 145 тестів у чотирьох файлах. Статус рецензування та Pull Request зазначено в розділах 7 і 8.

2. Таблиця аналізу вхідних умов для всіх функцій модуля

Класи еквівалентності об’єднують входи з однаковою очікуваною поведінкою. Межові значення перевіряють зміну правила: сусідні значення до порога, сам поріг і значення після нього. Для дискретного набору строків розстрочки перевірено сусідів кожного допустимого строку.

Таблиця 1 — Аналіз вхідних умов модуля order.ts

Функція	**Клас та представники входів**	**Межі й очікувана поведінка**
calcSubtotal	Непорожній масив, ціна ≥ 0, ціла кількість > 0; кілька позицій.	100 × 2 + 50 × 3 → 350; враховано кожну позицію.

Функція	**Клас та представники входів**	**Межі й очікувана поведінка**
calcSubtotal	Нульова ціна; від’ємна ціна; порожній масив.	price: −0,01 / 0 / 0,01 → NEGATIVE_PRICE / 0 / 0,01; [] → EMPTY_ORDER.
calcSubtotal	Кількість: додатна ціла; нуль, від’ємна, дробова, NaN, ±Infinity.	qty: 0 / 1 / 2 → INVALID_QTY / 100 / 200 для price = 100; 1,5 також відхиляється.
calcDiscountRate	basic, silver, gold; сума до порога та від порога.	Для 500 → 0 / 0,05 / 0,10; для 2000 → 0,05 / 0,10 / 0,15.
calcDiscountRate	Поріг великого замовлення; промокод і межа ставки.	1999 / 2000 / 2001 → 0 / 0,05 / 0,05 для basic; сумарні 0,24 / 0,25 / 0,30 → 0,24 / 0,25 / 0,25.
calcDiscountRate	Промокод відсутній, нульовий, додатний або від’ємний.	Відсутній → 0; −0,01 та −0,1 → INVALID_PROMO; ставка не перевищує 0,25.
calcShipping	Сума нижча за 1000 та від 1000.	999 / 1000 / 1001 → 60 / 0 / 0; уточнено межі 999,99 і 1000,01.
calcOrderTotal	Комбінації позицій, рівня та промокоду; округлення й доставка.	500 basic → 560; 1200 gold → 1080; 1050 gold → 1005, бо після знижки 945 < 1000.
calcOrderTotal	Поріг доставки саме після знижки 20 %.	Вхід 1249,99 / 1250 / 1250,01 → підсумок 1059,99 / 1000 / 1000,01; рівно 1000 дає безкоштовну доставку.

Функція	**Клас та представники входів**	**Межі й очікувана поведінка**
calcOrderTotal	Некоректні позиції або промокод.	Передає EMPTY_ORDER, NEGATIVE_PRICE, INVALID_QTY та INVALID_PROMO без приховування.
calcInstallment	Скінченна невід’ємна сума; строки 3, 6, 12.	1200 → 408 / 208 / 108. Сума −1 / 0 / 1 → INVALID_TOTAL / 0 / 0,34 для 3 місяців.
calcInstallment	Сусіди допустимих строків; дробовий або непідтримуваний строк.	2 / 3 / 4; 5 / 6 / 7; 11 / 12 / 13: приймається лише центральне значення. Інші → UNSUPPORTED_MONTHS.
calcInstallment	Грошове округлення; малі, великі та експоненційні числа.	0,749 / 0,750 / 0,751 на 3 місяці → 0,25 / 0,26 / 0,26; MIN_VALUE → 0; 10^{10} на 12 місяців → 900000000.
calcInstallment	Від’ємна сума, NaN, ±Infinity; хибні типи під час виконання; переповнення.	INVALID_TOTAL або UNSUPPORTED_MONTHS; 10^{21}, $1{,}2 \times 10^{21}$ та MAX_VALUE → INSTALLMENT_OUT_OF_RANGE.

У показовому прикладі збережено саме передбачені перевірки. calcShipping не викидає винятків для некоректних чисел; calcDiscountRate не перевіряє довільний рядок замість Tier, а calcSubtotal не перевіряє скінченність price. Такі додаткові обмеження методичкою не визначені, тому реалізацію прикладу не розширено. Це межі контракту, а не підстава приписувати функціям відсутню валідацію.

3. Лістинги розроблених тестів із поясненням добору вхідних даних

Лістинг 1 містить фактичний файл tests/order-example.test.ts. Порожні рядки лістингів опущено для компактності; виконувані рядки та очікувані числа взято з репозиторію. Для варіанта 12 повний набір тестів наведено в розділі 6.

Для calcSubtotal обрано кілька позицій і кількостей, порожній масив, ціну біля нуля та недопустимі кількості. Другу некоректну позицію перевірено окремо, щоб цикл не валідував лише перший елемент. Для calcDiscountRate перевірено всі рівні клієнта, поріг 1999/2000/2001, відсутність промокоду, від’ємний промокод і обмеження ставки 25 %.

Для calcShipping використано обидва класи та сусідів порога, зокрема суми з копійками. Для calcOrderTotal перевірено доставку після знижки, промокод за замовчуванням, кілька позицій, округлення та передавання всіх чотирьох винятків. Випадок 1050 для gold фіксує бізнес-правило методички: знижка переводить 945 грн нижче порога доставки, отже результат становить 1005 грн.

Параметризація it.each застосована до однорідних сценаріїв. Очікувані результати задано числами, незалежними від формули реалізації. toBeCloseTo використано для грошових сум і ставок, toBe — для цілочислового тарифу доставки. У toThrow передається функція, яка викликає модуль, щоб Vitest міг перехопити виняток. Перший тест явно поділено на Arrange — підготовка, Act — виклик, Assert — перевірка. Кожен тест створює власні дані та не залежить від порядку запуску.

Лістинг 1. Тести показового прикладу

```
import { describe, expect, it } from 'vitest';
import {
  calcSubtotal,
  calcDiscountRate,
  calcShipping,
```

```
  calcOrderTotal,
  type OrderItem,
  type Tier,
} from '../src/order';
const item = (price: number, qty = 1): OrderItem => ({
  sku: 'SKU-1',
  price,
  qty,
});
describe('calcSubtotal', () => {
  it('підсумовує позиції з урахуванням кількості', () => {
    // Arrange
    const items = [item(100, 2), { sku: 'SKU-2', price: 50, qty: 3 }];
    // Act
    const result = calcSubtotal(items);
    // Assert
    expect(result).toBeCloseTo(350, 2);
  });
  it('відхиляє порожнє замовлення', () => {
    expect(() => calcSubtotal([])).toThrow('EMPTY_ORDER');
  });
  it.each([
    [0, 0],
    [0.01, 0.01],
    [0.1, 0.1],
  ])('приймає ціну %s і повертає %s', (price, expected) => {
    expect(calcSubtotal([item(price)])).toBeCloseTo(expected, 6);
  });
  it.each([-1, -0.01])('відхиляє від’ємну ціну %s', (price) => {
    expect(() => calcSubtotal([item(price)])).toThrow('NEGATIVE_PRICE');
```

```
  });
  it.each([0, -1, 1.5, NaN, Infinity, -Infinity])(
    'відхиляє некоректну кількість %s',
    (qty) => {
      expect(() => calcSubtotal([item(100, qty)])).toThrow('INVALID_QTY');
    },
  );
  it.each([
    [1, 100],
    [2, 200],
  ])('для кількості %s повертає %s', (qty, expected) => {
    expect(calcSubtotal([item(100, qty)])).toBeCloseTo(expected, 2);
  });
  it('перевіряє також другу позицію замовлення', () => {
    expect(() => calcSubtotal([item(100), item(-0.01)])).toThrow(
      'NEGATIVE_PRICE',
    );
  });
});
describe('calcDiscountRate', () => {
  it.each<[Tier, number, number]>([
    ['basic', 500, 0],
    ['silver', 500, 0.05],
    ['gold', 500, 0.1],
    ['basic', 2000, 0.05],
    ['silver', 2000, 0.1],
    ['gold', 2000, 0.15],
  ])('рівень %s із сумою %s дає ставку %s', (tier, subtotal, expected) => {
    expect(calcDiscountRate(tier, subtotal)).toBeCloseTo(expected, 4);
  });
```

```
  it.each([
    [1999, 0],
    [2000, 0.05],
    [2001, 0.05],
  ])('межа великого замовлення %s дає %s', (subtotal, expected) => {
    expect(calcDiscountRate('basic', subtotal)).toBeCloseTo(expected, 4);
  });
  it.each([
    [0, 0.1],
    [0.14, 0.24],
    [0.15, 0.25],
    [0.2, 0.25],
  ])('промокод %s обмежує підсумкову ставку до %s', (promo, expected) => {
    expect(calcDiscountRate('gold', 500, promo)).toBeCloseTo(expected, 4);
  });
  it('обмежує сумарну знижку разом із бонусом великого замовлення', () => {
    expect(calcDiscountRate('gold', 2000, 0.5)).toBeCloseTo(0.25, 4);
  });
  it.each([-0.01, -0.1])('відхиляє від’ємний промокод %s', (promo) => {
    expect(() => calcDiscountRate('basic', 500, promo)).toThrow(
      'INVALID_PROMO',
    );
  });
});
describe('calcShipping', () => {
  it.each([
    [0, 60],
    [999, 60],
    [999.99, 60],
    [1000, 0],
```

```
    [1000.01, 0],
    [1001, 0],
  ])('сума %s визначає доставку %s', (amount, expected) => {
    expect(calcShipping(amount)).toBe(expected);
  });
});
describe('calcOrderTotal', () => {
  it.each<[number, Tier, number, number]>([
    [500, 'basic', 0, 560],
    [1200, 'gold', 0, 1080],
    [1050, 'gold', 0, 1005],
    [100, 'silver', 0, 155],
    [0, 'basic', 0, 60],
    [2000, 'gold', 0.5, 1500],
    [1250, 'basic', 0.2, 1000],
    [1249.99, 'basic', 0.2, 1059.99],
    [1250.01, 'basic', 0.2, 1000.01],
    [0.01, 'silver', 0, 60.01],
  ])(
    'сума %s, рівень %s, промокод %s дають підсумок %s',
    (price, tier, promo, expected) => {
      expect(calcOrderTotal([item(price)], tier, promo)).toBeCloseTo(
        expected,
        2,
      );
    },
  );
  it('використовує нульовий промокод за замовчуванням', () => {
    expect(calcOrderTotal([item(500)], 'basic')).toBeCloseTo(560, 2);
  });
```

```
  it('враховує кілька позицій та їх кількість', () => {
    expect(calcOrderTotal([item(100, 2), item(50, 3)], 'silver')).toBeCloseTo(
      392.5,
      2,
    );
  });
  it.each<[OrderItem[], number, string]>([
    [[], 0, 'EMPTY_ORDER'],
    [[item(-0.01)], 0, 'NEGATIVE_PRICE'],
    [[item(100, 0)], 0, 'INVALID_QTY'],
    [[item(100)], -0.01, 'INVALID_PROMO'],
  ])('передає виняток %s %s %s', (items, promo, code) => {
    expect(() => calcOrderTotal(items, 'gold', promo)).toThrow(code);
  });
});
```

4. Знімки екрана: результат прогону та таблиця покриття

Нижче наведено нові знімки реального PTY-термінала та згенерованого Vitest HTML-звіту після видалення тимчасового тесту. Термінальний вивід передавався до емулятора xterm без редагування змісту; знімки зроблено браузером автоматично. Первісні знімки з 91 тестом замінено, оскільки вони вже не відповідають завершеному модулю та фінальним 145 тестам.

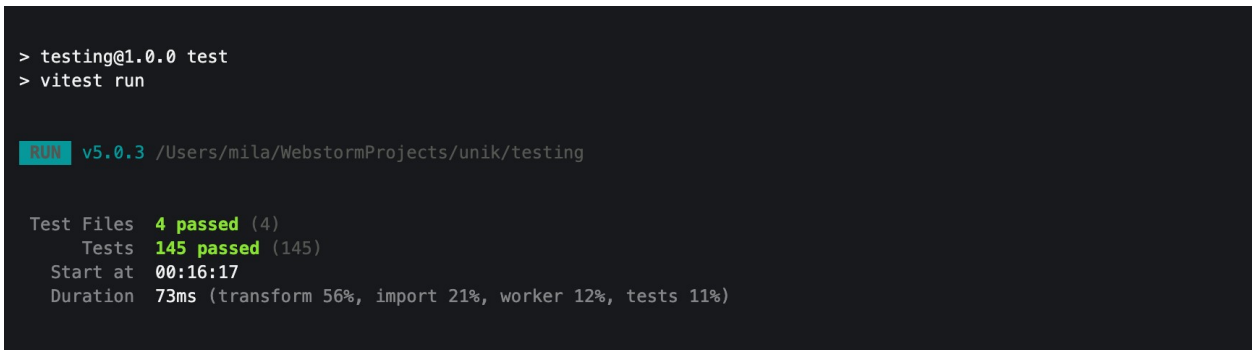


Рисунок 1 — Фінальний npm test: 145 успішних тестів, 4 файли

Збережено всі 79 тестів calcInstallment та 12 тестів попередньої лабораторної роботи: 8 для calculateAverage і 4 для celsiusToFahrenheit. Додано 54 тести показового прикладу. Підсумок узгоджується з кількістю 145 у журналі запуску.

Таблиця 2 — Фінальне покриття коду V8

Об’єкт	**Statements**	**Branches**	**Functions**	**Lines**
Увесь src	100 % 62/62	100 % 36/36	100 % 12/12	100 % 57/57
src/order.ts	100 % 51/51	100 % 28/28	100 % 9/9	100 % 47/47
calcInstallment	Окремо не обчислювали	100 % 8/8	Викликана	Рядки виконані

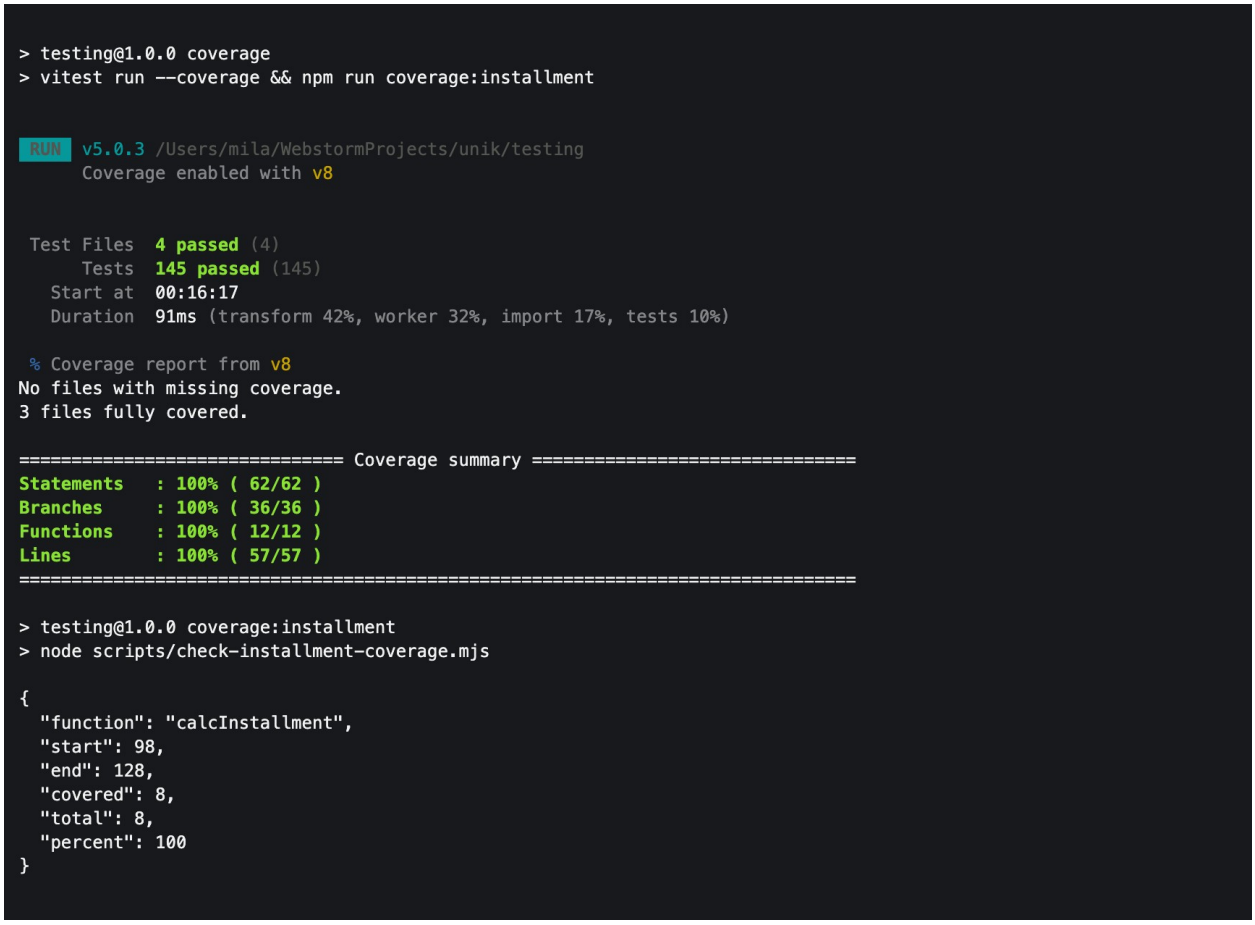
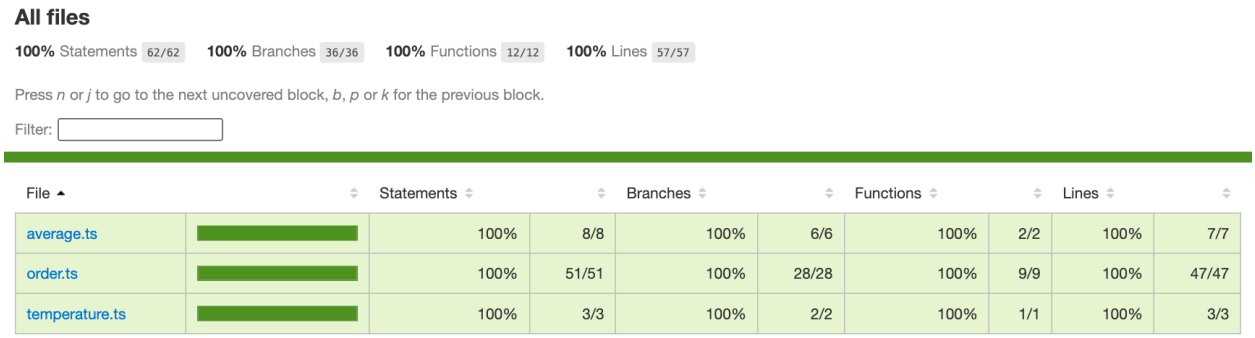


Рисунок 2 — Фінальний npm run coverage та окрема перевірка варіанта 12



All files

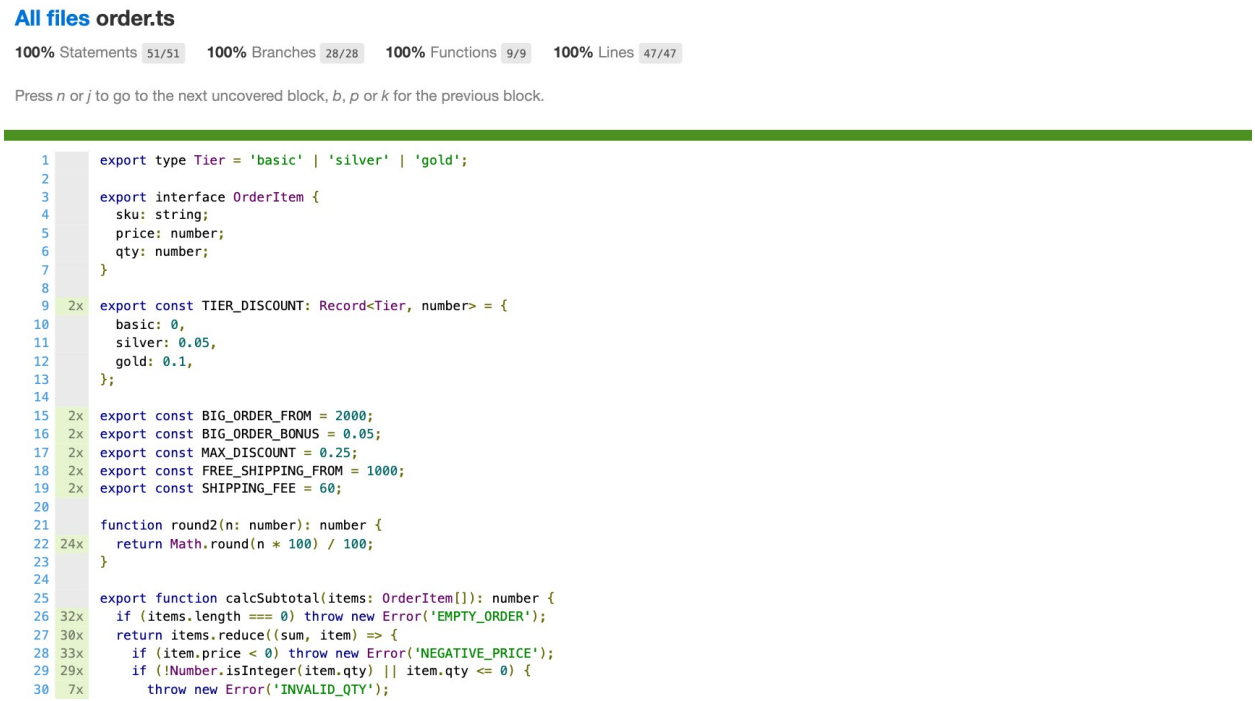
100% Statements 62/62 **100%** Branches 36/36 **100%** Functions 12/12 **100%** Lines 57/57

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
average.ts	100%	8/8	100%	6/6	100%	2/2	100%	7/7
order.ts	100%	51/51	100%	28/28	100%	9/9	100%	47/47
temperature.ts	100%	3/3	100%	2/2	100%	1/1	100%	3/3

Рисунок 3 — HTML Coverage Report із підсумками всіх модулів



All files order.ts

100% Statements 51/51 **100%** Branches 28/28 **100%** Functions 9/9 **100%** Lines 47/47

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1     export type Tier = 'basic' | 'silver' | 'gold';
2
3     export interface OrderItem {
4       sku: string;
5       price: number;
6       qty: number;
7     }
8
9   2x export const TIER_DISCOUNT: Record<Tier, number> = {
10      basic: 0,
11      silver: 0.05,
12      gold: 0.1,
13    };
14
15  2x export const BIG_ORDER_FROM = 2000;
16  2x export const BIG_ORDER_BONUS = 0.05;
17  2x export const MAX_DISCOUNT = 0.25;
18  2x export const FREE_SHIPPING_FROM = 1000;
19  2x export const SHIPPING_FEE = 60;
20
21    function round2(n: number): number {
22 24x   return Math.round(n * 100) / 100;
23    }
24
25    export function calcSubtotal(items: OrderItem[]): number {
26 32x   if (items.length === 0) throw new Error('EMPTY_ORDER');
27 30x   return items.reduce((sum, item) => {
28 33x     if (item.price < 0) throw new Error('NEGATIVE_PRICE');
29 29x     if (!Number.isInteger(item.qty) || item.qty <= 0) {
30  7x       throw new Error('INVALID_QTY');
```

Рисунок 4 — Покриття src/order.ts у фінальному HTML-звіті

HTML-звіт показує дев’ять покритих функцій order.ts з урахуванням допоміжних функцій, конструктора та callback. Відсоток гілок індивідуальної функції обчислено окремо, тому загальне покриття модуля не підміняє перевірку вимоги варіанта.

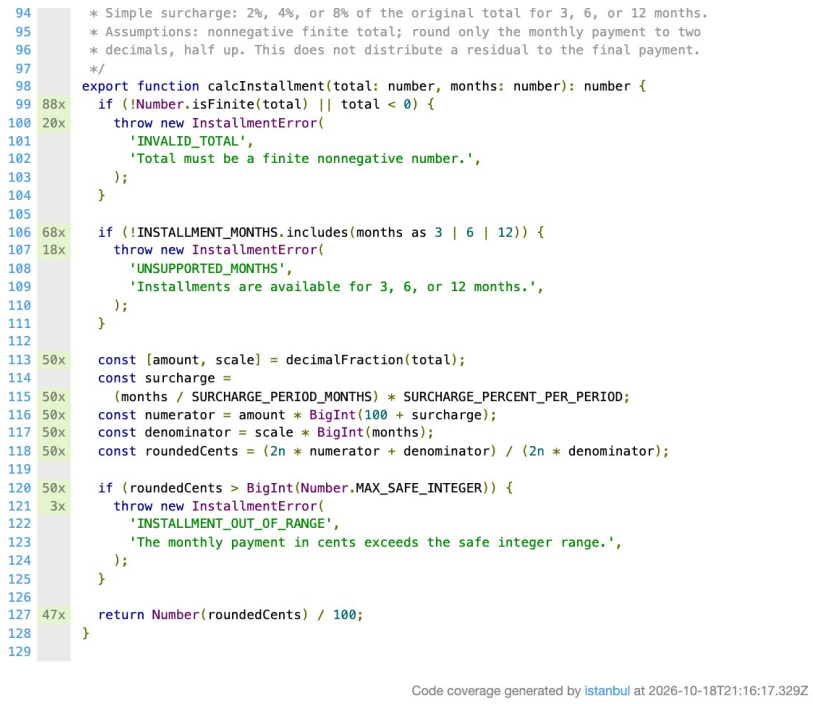


Рисунок 5 — Виконані рядки та лічильники calcInstallment у HTML-звіті



Рисунок 6 — Успішні ESLint, TypeScript і Prettier

Команди npm test, npm run coverage, npm run lint, npm run typecheck, npm run format:check та npm run coverage:installment завершилися з кодом 0. У coverage-конфігурації збережено пороги: Lines ≥ 80 %, Branches ≥ 80 %, Functions = 100 %. Окремий скрипт додатково вимагає Branches ≥ 90 % саме для calcInstallment.

5. Результат експерименту з тестом без твердження та висновок щодо метрики покриття

Експеримент виконано саме для calcOrderTotal, як указано в кроці 6 методички. Спочатку виконано npm run coverage без додаткового тесту. Потім

до tests/order-example.test.ts тимчасово додано наведений нижче блок, повторено ту саму команду й збережено обидва JSON-звіти та знімки термінала. Після цього блок видалено, файл відновлено, покриття та інші перевірки виконано повторно.

Лістинг 2. Тимчасовий тест без твердження

```
it('виклик без перевірки результату', () => {
  calcOrderTotal([item(100)], 'silver');
});
```

Таблиця 3 — Порівняння експериментальних вимірювань для всього src

Показник	**До тесту**	**Після додавання**	**Після видалення**
Тести	145	146	145
Statements	100 % (62/62)	100 % (62/62)	100 % (62/62)
Branches	100 % (36/36)	100 % (36/36)	100 % (36/36)
Functions	100 % (12/12)	100 % (12/12)	100 % (12/12)
Lines	100 % (57/57)	100 % (57/57)	100 % (57/57)

Зміна кожної з чотирьох метрик становить 0 відсоткових пунктів. Модуль order.ts також залишився на 100 %: Statements 51/51, Branches 28/28, Functions 9/9, Lines 47/47. Додавання тесту не відкрило нових шляхів виконання, оскільки ці шляхи вже були покриті тестами з перевірками.

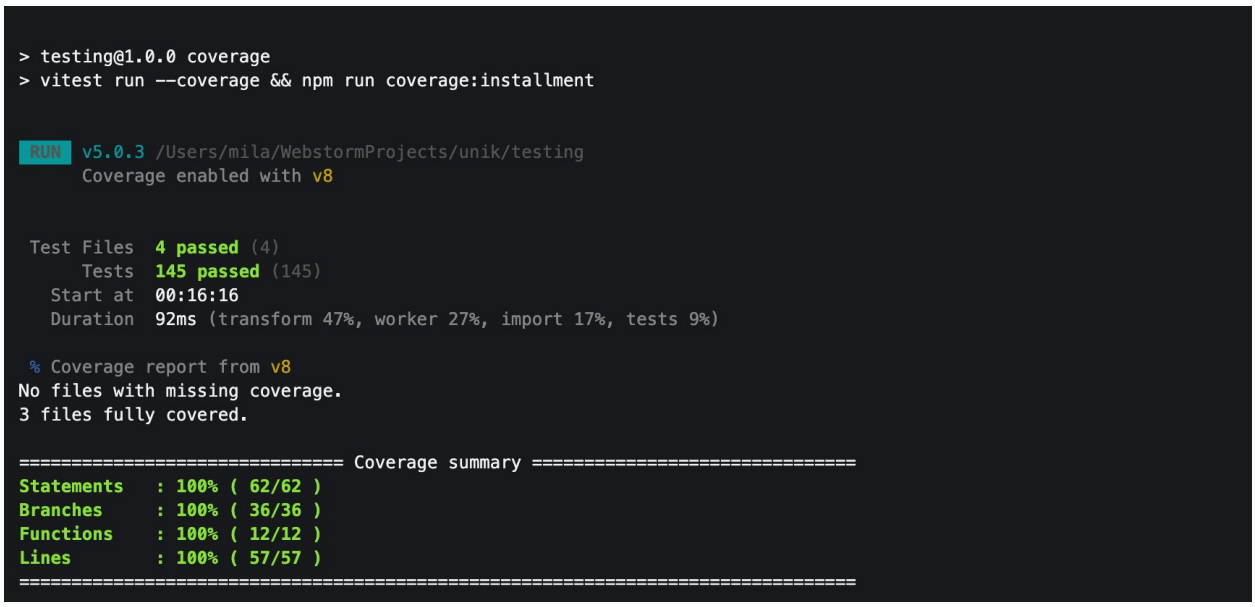


Рисунок 7 — Експеримент до додавання тесту: 145 тестів, усі метрики 100 %

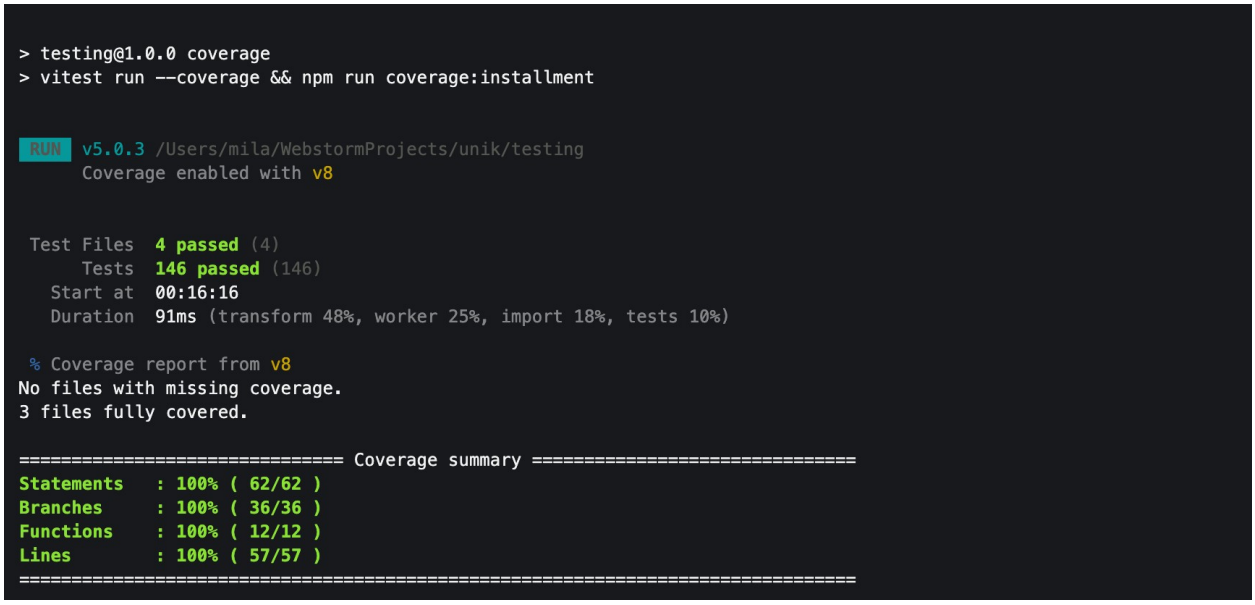


Рисунок 8 — Експеримент після додавання тесту: 146 тестів, покриття незмінне

Тест без expect виконує код і виявить необроблений виняток, однак не звіряє повернену суму з очікуваними 155 грн. Якщо функція поверне інше число без винятку, такий тест усе одно пройде. Метрика покриття враховує виконання інструкцій, гілок, функцій і рядків, а не правильність тверджень. За наявності невиконаного шляху порожній виклик міг би підвищити показник без перевірки результату; у цьому експерименті зростання фактично не було.

Отже, 100 % покриття є свідченням виконаних шляхів у цьому наборі даних, але не доказом відсутності дефектів. Якість тестів забезпечують незалежні очікувані значення, змістовні матчери, перевірки винятків та обґрунтовані класи входів. Фінальний набір не містить тимчасового тесту. Для повторення досліду додано команду npm run experiment:order; попередній experiment:installment є лише додатковим дослідом і не підміняє вимогу методички.

6. Специфікація, реалізація та тести функції за індивідуальним варіантом

Специфікація та прийняті припущення

Явна вимога варіанта 12: calcInstallment(total, months) повертає місячний платіж для 3, 6 та 12 місяців із надбавкою 2 % за кожні три місяці. Загальні вимоги до варіантів передбачають параметризовані тести, перевірку сусідів межі, виняток із власним кодом помилки, явні числові очікування й покриття гілок функції не нижче 90 %.

У наявній реалізації прийнято просту надбавку до початкової суми: 2 % для 3 місяців, 4 % для 6 та 8 % для 12. Розрахунок має зміст total × (1 + 0,02 × months / 3) / months. Для 1200 грн відповідні платежі становлять 408, 208 і 108 грн. Нарахування не є складним відсотком. Цей спосіб тлумачення надбавки збережено як припущення реалізації.

Додаткові припущення: total має бути скінченним числом ≥ 0; нуль дозволений; строк має точно належати множині {3, 6, 12}; результат округлюється до двох десяткових знаків за правилом half up. Округлюється місячний платіж, залишок копійок не переноситься на останній місяць. Зокрема, 0,01 грн на 3 місяці дає 0 грн після округлення. Методичка не задає окремих правил для цих випадків, тому вони не подаються як її дослівні вимоги.

decimalFraction розбирає десятковий запис числа та виконує проміжні дії з bigint, щоб не спотворювати пороги округлення через додаткові двійкові

похибки. Вхідний number уже має властиву JavaScript точність; алгоритм працює з його фактичним десятковим записом. При перевищенні безпечного цілого діапазону платежу в копійках реалізація відхиляє розрахунок.

Таблиця 4 — Валідація та коди помилок індивідуальної функції

Код	**Причина**	**Приклад перевірки**
INVALID_TOTAL	Від’ємний або нескінченний total, NaN, хибний тип у runtime.	−1; NaN; ±Infinity; рядок; null; undefined; {}; bigint.
UNSUPPORTED_ MONTHS	Строк поза {3, 6, 12}; дробовий, нескінченний або хибний тип.	0; 9; 24; 3,5; NaN; ±Infinity; «3»; null; undefined.
INSTALLMENT_OUT_ OF_RANGE	Округлений платіж у копійках перевищує MAX_ SAFE_INTEGER.	10^{21}; $1{,}2 \times 10^{21}$; Number.MAX_ VALUE.

Власний клас InstallmentError розширює Error та має поле code. Якщо обидва аргументи некоректні, спочатку перевіряється total. Це зафіксовано окремим тестом. Реалізацію calcInstallment та її 79 наявних тестів збережено без змін, оскільки перевірка за отриманою методичкою не виявила необхідності їх виправляти.

Лістинг 3. Реалізація варіанта 12 із src/order.ts

```
export const INSTALLMENT_MONTHS = [3, 6, 12] as const;
export const SURCHARGE_PERCENT_PER_PERIOD = 2;
export const SURCHARGE_PERIOD_MONTHS = 3;
export type InstallmentErrorCode =
  'INVALID_TOTAL' | 'UNSUPPORTED_MONTHS' |
'INSTALLMENT_OUT_OF_RANGE';
export class InstallmentError extends Error {
  constructor(
```

```
    public readonly code: InstallmentErrorCode,
    message: string,
  ) {
    super(message);
    this.name = 'InstallmentError';
  }
}
// Interpret the public number's decimal representation without binary rounding.
function decimalFraction(value: number): [bigint, bigint] {
  const [coefficient, exponent = '0'] = value.toString().split('e');
  const [whole, fraction = ''] = coefficient.split('.');
  const digits = BigInt(whole + fraction);
  const scale = fraction.length - Number(exponent);
  if (scale > 0) {
    return [digits, 10n ** BigInt(scale)];
  }
  return [digits * 10n ** BigInt(-scale), 1n];
}
/**
 * Simple surcharge: 2%, 4%, or 8% of the original total for 3, 6, or 12 months.
 * Assumptions: nonnegative finite total; round only the monthly payment to two
 * decimals, half up. This does not distribute a residual to the final payment.
 */
export function calcInstallment(total: number, months: number): number {
  if (!Number.isFinite(total) || total < 0) {
    throw new InstallmentError(
      'INVALID_TOTAL',
      'Total must be a finite nonnegative number.',
    );
  }
```

```
  if (!INSTALLMENT_MONTHS.includes(months as 3 | 6 | 12)) {
    throw new InstallmentError(
      'UNSUPPORTED_MONTHS',
      'Installments are available for 3, 6, or 12 months.',
    );
  }
  const [amount, scale] = decimalFraction(total);
  const surcharge =
    (months / SURCHARGE_PERIOD_MONTHS) *
SURCHARGE_PERCENT_PER_PERIOD;
  const numerator = amount * BigInt(100 + surcharge);
  const denominator = scale * BigInt(months);
  const roundedCents = (2n * numerator + denominator) / (2n * denominator);
  if (roundedCents > BigInt(Number.MAX_SAFE_INTEGER)) {
    throw new InstallmentError(
      'INSTALLMENT_OUT_OF_RANGE',
      'The monthly payment in cents exceeds the safe integer range.',
    );
  }
  return Number(roundedCents) / 100;
}
```

Пояснення добору тестових даних

Позитивні таблиці містять усі три строки для сум 1200 і 3000 та нуль для кожного строку. Окрема таблиця перевіряє малі дроби, експоненційний запис, MIN_VALUE і велику допустиму суму. Значення 0,749 / 0,750 / 0,751 віддалені на одну тисячну від межі округлення; 0,750 × 1,02 / 3 = 0,255, тому саме середнє значення переходить до 0,26.

Правило «межа мінус один, межа, межа плюс один» застосовано до 3, 6 і 12 місяців та до нижньої межі суми: −1, 0, 1. Набори NaN, нескінченностей і

хибних runtime-типів перевіряють відхилення входів, а великі суми перевіряють виняток безпечного діапазону. Очікувані платежі задано літералами. toBeCloseTo застосовано для дробів; toBe — також у збережених тестах з точними еталонними числами та нулем. toThrow(InstallmentError) перевіряє клас, а toThrow(expect.objectContaining(...)) — власний код і, для INVALID_TOTAL, повідомлення.

Лістинг 4. Повний файл tests/order.test.ts

```
import { describe, expect, it } from 'vitest';
import { calcInstallment, InstallmentError } from '../src/order';
describe('Розрахунок місячного платежу calcInstallment', () => {
  it.each([
    [1200, 3, 408],
    [1200, 6, 208],
    [1200, 12, 108],
    [3000, 3, 1020],
    [3000, 6, 520],
    [3000, 12, 270],
  ])('для суми %s і строку %s повертає %s', (total, months, expected) => {
    // Arrange: expected is an independent literal reference value.
    // Act
    const payment = calcInstallment(total, months);
    // Assert
    expect(payment).toBe(expected);
  });
  it.each([3, 6, 12])(
    'для нульової суми й строку %s повертає нуль',
    (months) => {
      expect(calcInstallment(0, months)).toBe(0);
    },
  );
```

```
it.each([
  [0.01, 3, 0],
  [0.02, 3, 0.01],
  [1, 3, 0.34],
  [100, 6, 17.33],
  [100, 12, 9],
  [0.1 + 0.2, 3, 0.1],
  [1.004, 6, 0.17],
  [1.005, 6, 0.17],
  [1.006, 6, 0.17],
  [1.125, 3, 0.38],
  [1.25, 12, 0.11],
  [1.249, 12, 0.11],
  [1.251, 12, 0.11],
  [0.1249, 12, 0.01],
  [0.125, 12, 0.01],
  [0.1251, 12, 0.01],
  [0.3749, 12, 0.03],
  [0.375, 12, 0.03],
  [0.3751, 12, 0.03],
  [1.499, 3, 0.51],
  [1.5, 3, 0.51],
  [1.501, 3, 0.51],
  [1e-7, 3, 0],
  [1.2e-7, 6, 0],
  [Number.MIN_VALUE, 12, 0],
  [1e10, 12, 900000000],
])(
  'для суми %s і строку %s округлює платіж до %s',
  (total, months, expected) => {
```

```
    expect(calcInstallment(total, months)).toBeCloseTo(expected, 10);
  },
);
it.each([
  [0.124, 0.04],
  [0.125, 0.04],
  [0.126, 0.04],
  [0.749, 0.25],
  [0.75, 0.26],
  [0.751, 0.26],
])('для суми %s округлює місячний платіж до %s', (total, expected) => {
  expect(calcInstallment(total, 3)).toBeCloseTo(expected, 10);
});
it.each([
  [2, false],
  [3, true],
  [4, false],
  [5, false],
  [6, true],
  [7, false],
  [11, false],
  [12, true],
  [13, false],
])('для граничного строку %s допустимість дорівнює %s', (months, valid) => {
  if (valid) {
    expect(calcInstallment(0, months)).toBe(0);
  } else {
    expect(() => calcInstallment(0, months)).toThrow(InstallmentError);
  }
});
```

```
it.each([-1, NaN, Infinity, -Infinity, '100', null, undefined, {}, 100n])(
  'відхиляє некоректну суму %s з кодом INVALID_TOTAL',
  (total) => {
    const calculate = () => calcInstallment(total as number, 3);
    expect(calculate).toThrow(InstallmentError);
    expect(calculate).toThrow(
      expect.objectContaining({
        code: 'INVALID_TOTAL',
        message: 'Total must be a finite nonnegative number.',
      }),
    );
  },
);
it.each([
  -1,
  0,
  1,
  9,
  24,
  3.5,
  NaN,
  Infinity,
  -Infinity,
  '3',
  null,
  undefined,
])(
  'відхиляє непідтримуваний строк %s з кодом UNSUPPORTED_MONTHS',
  (months) => {
    expect(() => calcInstallment(1200, months as number)).toThrow(
```

```
        expect.objectContaining({ code: 'UNSUPPORTED_MONTHS' }),
      );
    },
  );
  it.each([
    [-1, 'INVALID_TOTAL'],
    [0, null],
    [1, null],
  ])('перевіряє суму %s біля нижньої межі', (total, code) => {
    if (code) {
      expect(() => calcInstallment(total, 3)).toThrow(
        expect.objectContaining({ code }),
      );
    } else {
      expect(calcInstallment(total, 3)).toBe(total === 0 ? 0 : 0.34);
    }
  });
  it('приймає від’ємний нуль і повертає додатний нуль', () => {
    expect(calcInstallment(-0, 3)).toBe(0);
  });
  it('спочатку повідомляє про некоректну суму, якщо обидва аргументи хибні',
() => {
    expect(() => calcInstallment(-1, 9)).toThrow(
      expect.objectContaining({ code: 'INVALID_TOTAL' }),
    );
  });
  it.each([1e21, 1.2e21, Number.MAX_VALUE])(
    'відхиляє суму %s, коли платіж у копійках перевищує безпечний діапазон',
    (total) => {
      expect(() => calcInstallment(total, 3)).toThrow(
```

```
      expect.objectContaining({ code: 'INSTALLMENT_OUT_OF_RANGE' }),
    );
  },
 );
});
```

Окрема перевірка покриття гілок

Команда npm run coverage:installment читає coverage/coverage-final.json, знаходить calcInstallment у fnMap та відбирає лише branchMap-лічильники всередині її діапазону рядків 98–128. Отримано 8 покритих лічильників із 8, тобто 100 %, що перевищує необхідні 90 %. Загальні Branches 28/28 для order.ts та 36/36 для всіх модулів наведено окремо. Допоміжна decimalFraction знаходиться поза діапазоном calcInstallment, а її гілки враховано в покритті всього модуля.

```
> testing@1.0.0 coverage:installment
> node scripts/check-installment-coverage.mjs

{
  "function": "calcInstallment",
  "start": 98,
  "end": 128,
  "covered": 8,
  "total": 8,
  "percent": 100
}
```

Рисунок 9 — Окрема перевірка calcInstallment: 8/8 гілок, 100 %

Скрипт припиняє виконання з помилкою, якщо файл, функція або лічильники відсутні чи результат нижчий за 90 %. Тому перевірка не зводиться до загального відсотка модуля. Первинні JSON-лічильники та журнал окремої команди збережено в output/lab2/evidence.

7. Текст рецензії на роботу однокурсника

Статус: очікує виконання на основі реальної роботи однокурсника. Посилання на Pull Request однокурсника не надано; підтвердженого доступу до такої роботи немає. Рецензію не виконано, коментарі не опубліковано. Вигадані зауваження до невідомого коду не додаються.

Для завершення потрібно отримати доступний URL Pull Request однокурсника, прочитати реалізацію та тести і сформулювати щонайменше три змістовні коментарі з посиланнями на конкретні рядки. Перевіряються наявність межових тестів, незалежність очікувань від продуктової формули та відповідність матчерів типам даних. Також потрібно оцінити назви тестів, зміст тверджень і незалежність тестів. Після фактичного рецензування до цього розділу слід додати справжній текст зауважень та посилання на коментарі.

8. Посилання на репозиторій і запит на злиття

Репозиторій: https://github.com/milunchik/testing

Фактичний origin для fetch і push: git@github.com:milunchik/testing.git. Поточна гілка: feature/lab2-vitest-installment. Початковий HEAD: 7e03d07bdf0c79b4cf3e0838ebfae55b1c1d544f. Перевірену реалізацію та 54 нові тести опубліковано комітом 25fcb2080128cb52a4c9efb701fc729102f71c46; фінальні матеріали звіту додано окремим комітом тієї самої гілки.

Перша перевірка SSH завершилася повідомленням «Permission denied (publickey)». Додаткова перевірка HTTPS підтвердила авторизацію облікового запису milunchik і право push. Гілку опубліковано через HTTPS без зміни origin; через GitHub API створено відкритий Pull Request №2.

Pull Request: https://github.com/milunchik/testing/pull/2

Базова гілка PR — feature/calculate-average: вона містить попередню лабораторну роботу у власному PR №1. Така база відокремлює зміни лабораторної №2 та зберігає попередні тести. Злиття PR №1 або №2 не виконувалося. Підтверджені GitHub-дані, статус гілки та URL збережено в output/lab2/evidence/github-access.json і pull-request.json. Єдиний незавершений зовнішній етап — рецензія на реальний PR однокурсника.

9. Висновки

Під час лабораторної роботи доповнено модуль розрахунку замовлення чотирма функціями показового прикладу відповідно до оригінальної методички та перевірено індивідуальну функцію calcInstallment для варіанта 12. Тестові дані дібрано за класами еквівалентності, межовими значеннями й некоректними входами; використано параметризовані тести, незалежні числові еталони та перевірки винятків. Усі 145 тестів пройшли, зокрема збережені тести попередньої лабораторної роботи. Покриття коду становить 100 % за чотирма метриками; окреме покриття гілок calcInstallment — 100 % (8/8). Перевірки ESLint, TypeScript і Prettier завершилися успішно.

Експеримент із викликом calcOrderTotal без твердження тимчасово збільшив кількість тестів до 146, але не змінив покриття. Це підтвердило, що виконання коду не перевіряє правильність результату без змістовного твердження. Тимчасовий тест видалено, фінальний стан перевірено повторно. Звіт оновлено реальними доказами виконання, а зміни опубліковано в Pull Request №2. Рецензування однокурсника залишається незавершеним зовнішнім етапом і потребує доступу до його справжнього Pull Request.